



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE



Fundamentals of Software Architecture: A Modern Engineering Approach

Mark Richards and Neal Ford—hands-on practitioners who have taught software architecture classes professionally for years—focus on architecture principles that apply across all technology stacks. You'll explore software architecture in a modern light, taking into account all the innovations of the past decade.

O'REILLY®

Second
Edition



Fundamentals of Software Architecture

A Modern Engineering Approach

Mark Richards & Neal Ford





Documentação de Arquitetura com o C4 Model

Nesta aula, vamos documentar a arquitetura de um sistema real, o Compasso, em quatro níveis de zoom com o C4 Model: do contexto ao código.



C4 Model: desenhando a arquitetura do nosso projeto

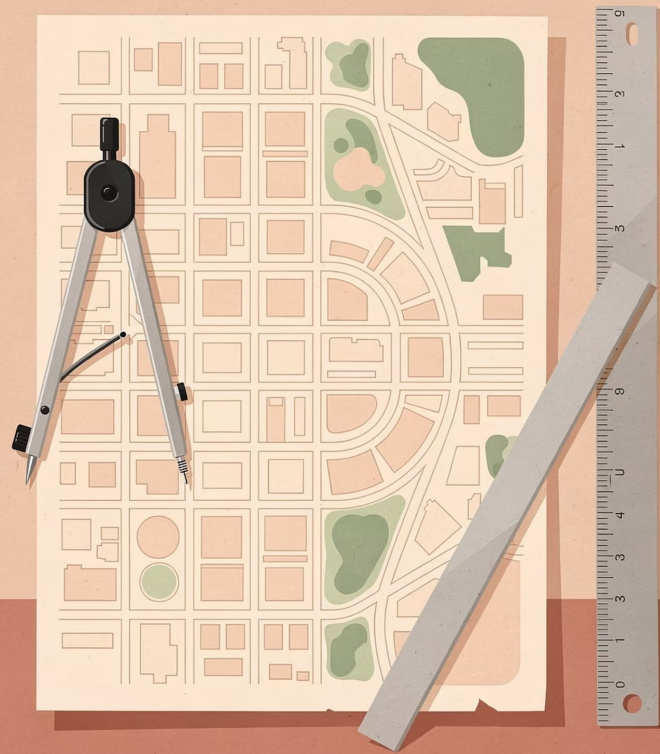
A ponte entre o conceitual e o prático, aprenda a documentar sua arquitetura de forma visual, clara e versionável.



Por que documentar arquitetura?

"Decisões esquecidas se repetem. Decisões mal comunicadas geram retrabalho."

Documentar a arquitetura **não é criar um contrato imutável**, é criar um mapa compartilhado que a equipe pode consultar e atualizar. O mapa muda conforme o sistema cresce, mas sem mapa, ninguém sabe onde está nem para onde ir.



A metáfora do mapa urbano

O que Garcia diz

"Documentar a arquitetura é como fazer um mapa de uma cidade que está sendo construída. O mapa muda conforme a cidade cresce, mas sem mapa, ninguém sabe onde está nem para onde ir."

Documentação arquitetural não é um contrato, é uma **ferramenta de comunicação.**

Aplique ao seu projeto

Imagine que duas pessoas da equipe começam a codar ao mesmo tempo:

Uma cria o backend na pasta `server /`

Outra cria na pasta `api /`

Sem combinarem a estrutura antes, teriam retrabalho logo no **primeiro dia**. O C4 é a forma de combinar.



Respostas que você provavelmente deu

Quando perguntamos "por que documentamos a arquitetura se ela pode mudar?", as respostas mais comuns são — e todas são válidas:



"Para lembrar"

A memória coletiva da equipe tem limite. Documentação externaliza decisões importantes.



"Para os outros entenderem"

Novos membros na equipe precisam de contexto, diagramas aceleram o onboarding.



"Para o stakeholder"

O parceiro de projeto precisa entender o que está sendo construído sem conhecer o código.



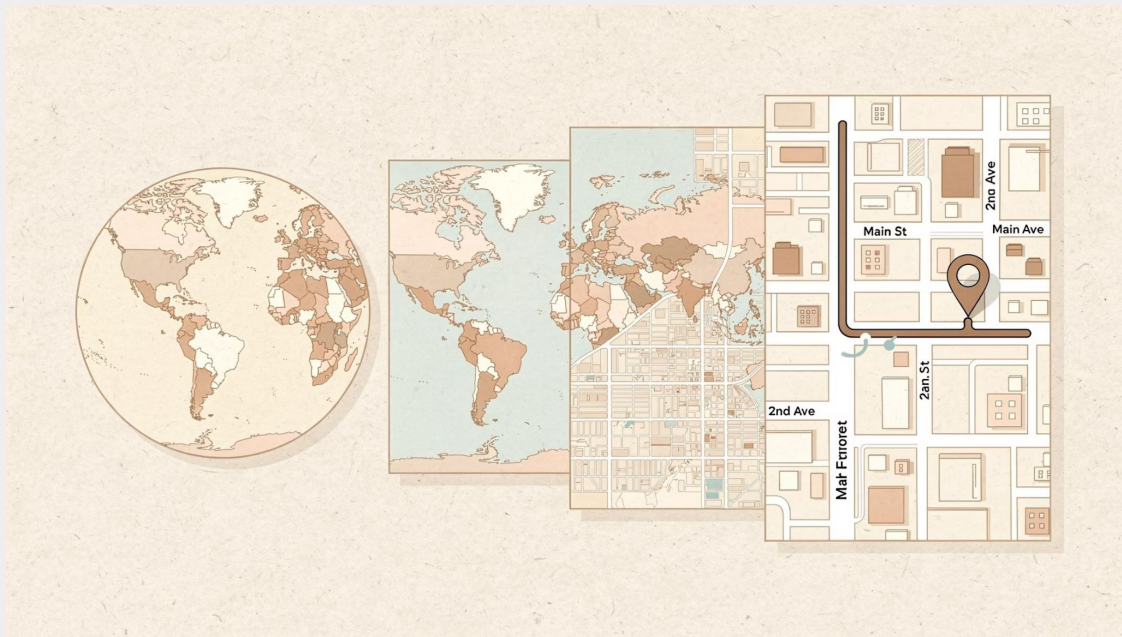
Duas palavras que o C4 usa de forma diferente

No C4	O que significa	O que NÃO significa	No Compasso
Contêiner	Qualquer processo em execução independente: uma aplicação ou um banco de dados.	NÃO é um contêiner Docker.	Frontend (React/Vite), API (Node.js/Express) e banco (SQLite).
Componente	Um módulo dentro de um contêiner C4, com uma responsabilidade clara.	NÃO é um componente React.	Dentro da API: TimeEntryController, TimeEntryService, TimeEntryRepository, ProjectService e UserService.

Mesmo sem Docker, o Compasso tem três contêineres C4. E a API não tem nenhum componente React, mas tem cinco componentes C4.



C4 Model: Visualizando Arquiteturas com Hierarquia



Criado por **Simon Brown** e inspirado na metáfora do Google Maps: você escolhe o nível de zoom adequado para o seu público.



Contexto

O sistema no mundo



Contêineres

Unidades de deploy



Componentes

Estrutura interna

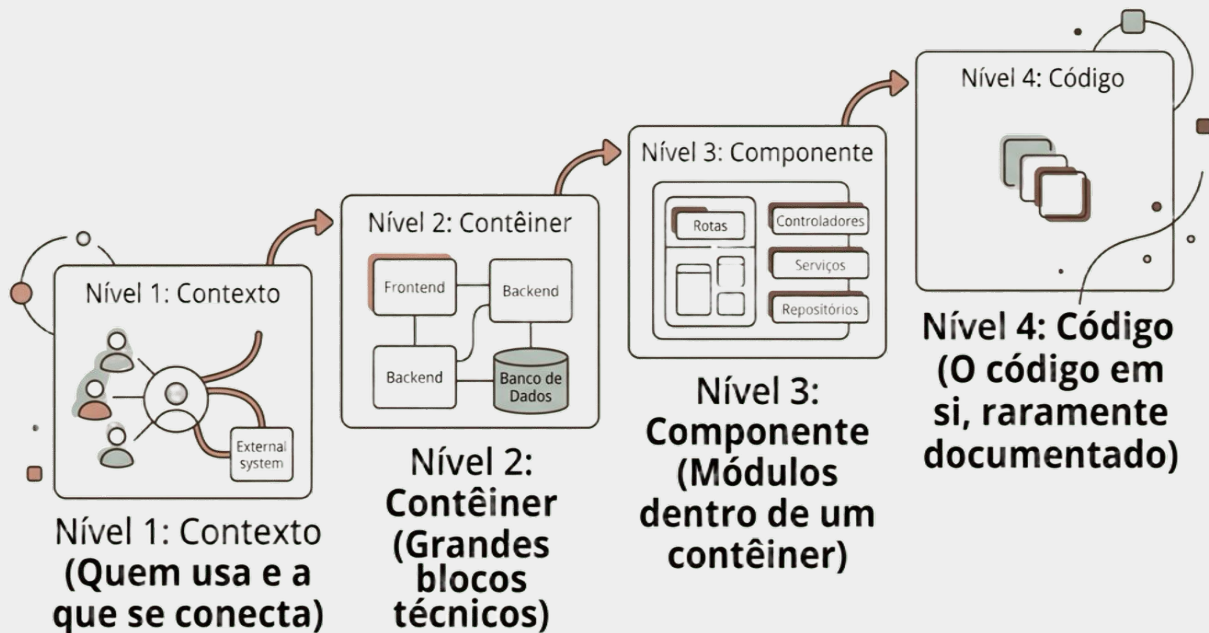


Código

Detalhes de impl.

O C4 Model: quatro níveis de zoom

O C4 Model organiza a documentação arquitetural em níveis progressivos de detalhe — como dar zoom em um mapa. Cada nível responde a uma pergunta diferente sobre o sistema.



Contexto: o ponto de partida



O diagrama de Contexto responde à pergunta mais fundamental: **quem usa o sistema e com o que ele se conecta?** Sem nenhum detalhe técnico, é a visão do stakeholder.



Nível 1 — Diagrama de Contexto

Pergunta central: quem usa o sistema e com o que ele se conecta?

O que aparece

Usuários (pessoas), o sistema que você constrói, e sistemas externos com os quais ele se integra. Nenhum detalhe técnico.

Exemplo: Compasso

Colaborador → Compasso ← Gestor

Sem sistema externo nesta versão.

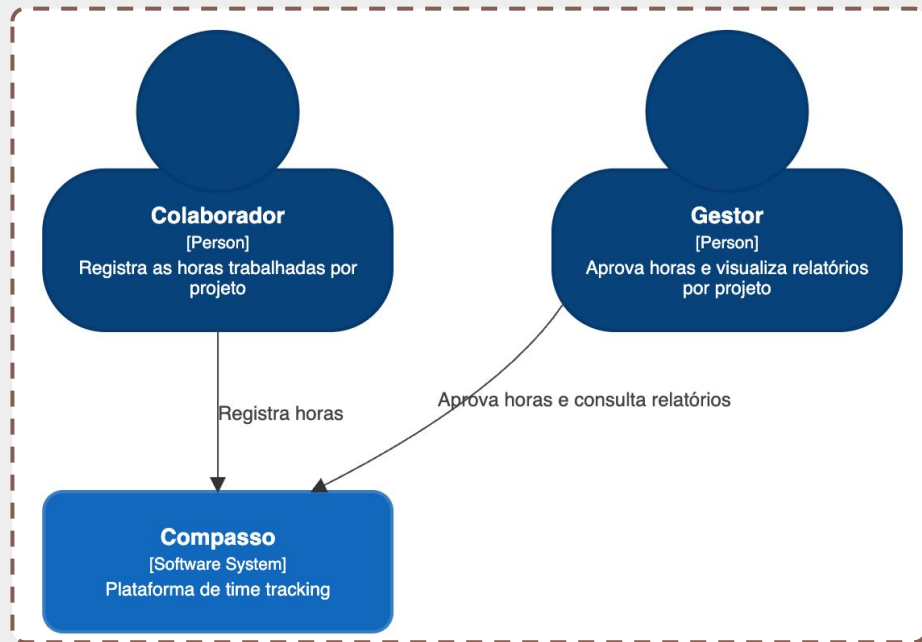
Para quem serve

É o diagrama que você mostra ao **parceiro na Sprint Review**.

Qualquer pessoa entende sem saber programar.



Exemplo: Compasso — Contexto



Nenhum detalhe técnico aparece neste nível. Backend, banco, linguagem — tudo invisível.

O que mostrar aqui

- Atores externos (usuários, sistemas)
- O sistema que você está construindo (uma única caixa)
- Conexões com APIs ou serviços externos

Para quem serve

Este é o diagrama que você mostra ao **parceiro na Sprint Review**. Qualquer pessoa consegue entender sem conhecimento técnico.



Compasso: lendo o Diagrama de Contexto

O Compasso é uma plataforma simples de time tracking: registra as horas trabalhadas por projeto.

Uma caixa central: o sistema Compasso.

- Dois atores: o Colaborador (registra horas) e o Gestor (aprova horas e vê relatórios por projeto).
- Nenhum sistema externo nesta versão: só pessoas e o sistema.

Para pensar

E se o Compasso passasse a avisar o Gestor por e-mail?

Uma nova caixa apareceria no diagrama: um sistema externo de e-mail, ligado ao Compasso. Nada mais mudaria no Nível 1.



Contêiner: abrindo a caixa do sistema



No Nível 2, damos zoom no sistema e revelamos os grandes blocos técnicos. Aqui aparecem as tecnologias, os processos em execução e como eles se comunicam entre si.



Nível 2 — Diagrama de Contêiner

Agora **abrimos a caixa** do sistema e mostramos os grandes blocos técnicos. Cada contêiner é um processo separado (aplicação, banco de dados, serviço).

Exemplo prático

Para o Compasso:

Frontend — React/Vite

API — Node.js/Express

Banco de Dados — SQLite

Para cada contêiner, sempre indique a tecnologia utilizada. Lembre: contêiner C4 não é Docker.

Reflexão para sua equipe

Quais contêineres o projeto de vocês vai ter?

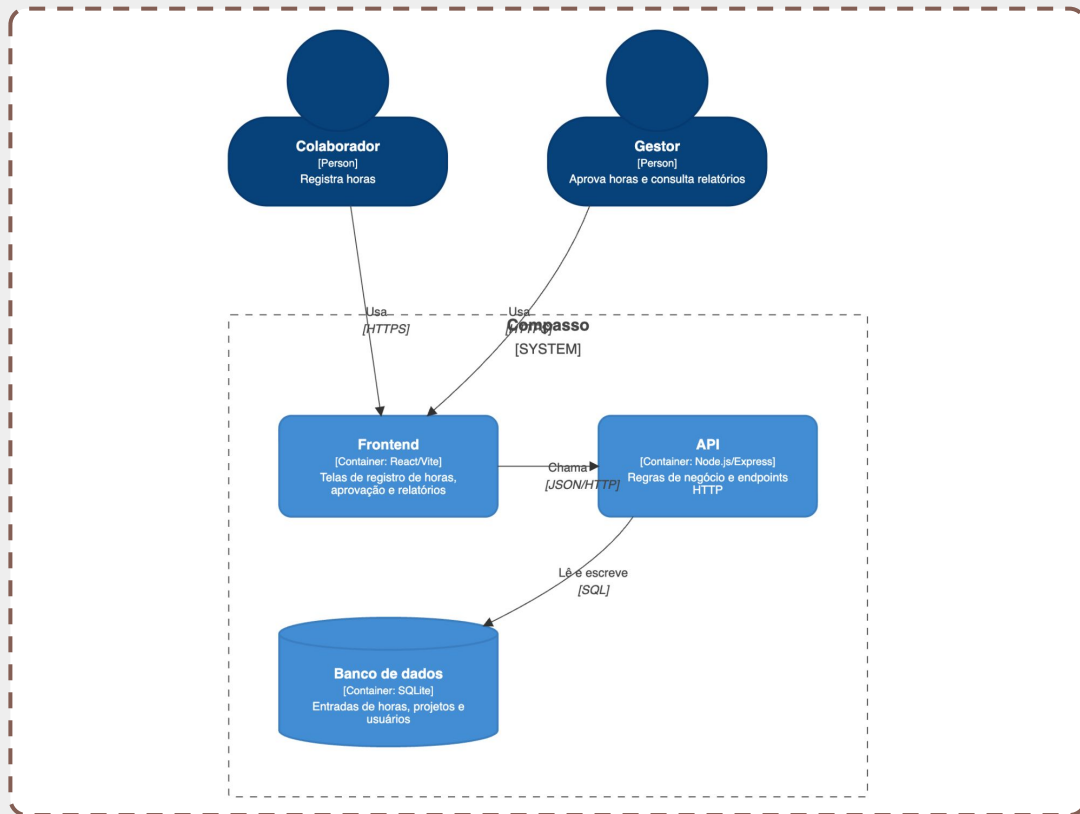
- É o padrão Frontend + Backend + Banco?
- Há algum serviço externo integrado?
- Existe algum contêiner adicional (cache, fila)?

❏ O Nível 2 é o diagrama mais útil no dia a dia — ele mostra a estrutura sem entrar em detalhes de código.



Exemplo: Compasso — Contêineres

Cada contêiner tem uma tecnologia explícita. Pergunte à sua equipe: "*Quais contêineres o nosso projeto vai ter? É o mesmo padrão (frontend, backend, banco) ou tem algo diferente?*"



Compasso: o Diagrama de Contêineres (O Mapa da Cidade)

Para o Compasso, o Diagrama de Contêineres revela:

- Frontend (React/Vite): roda no navegador e é o que o Colaborador e o Gestor usam.
- API (Node.js/Express): um único backend, organizado em camadas.
- Banco de dados (SQLite): sem servidor de banco para administrar.
- Um sistema maior poderia ter vários microsserviços aqui. O Compasso tem uma API só, e essa escolha está registrada no ADR-002.

ADR-002: Usar monolito modular

Status: Aceito

Contexto: equipe de 4 pessoas, prazo de 4 semanas, sem necessidade de escala horizontal.

Decisão: vamos construir um monolito modular (Node.js/Express com camadas separadas), porque a equipe é pequena e o prazo é curto.

Consequências:

Mais fácil: um único deploy, simples de desenvolver e testar.

Mais difícil: migrar para microsserviços no futuro será custoso. Aceito para este contexto.



Nível 3 — Diagrama de Componente

Entramos dentro de um contêiner, geralmente o Backend API, e mostramos os módulos que o compõem. Este nível conecta diretamente com a organização de pastas discutida nas aulas anteriores.

1

Routes

Recebe a requisição HTTP (ex.: POST /time-entry)

2

Controllers

Orquestra a lógica de entrada (ex.: TimeEntryController)

3

Services

Contém as regras de negócio (ex.: TimeEntryService)

4

Repositories

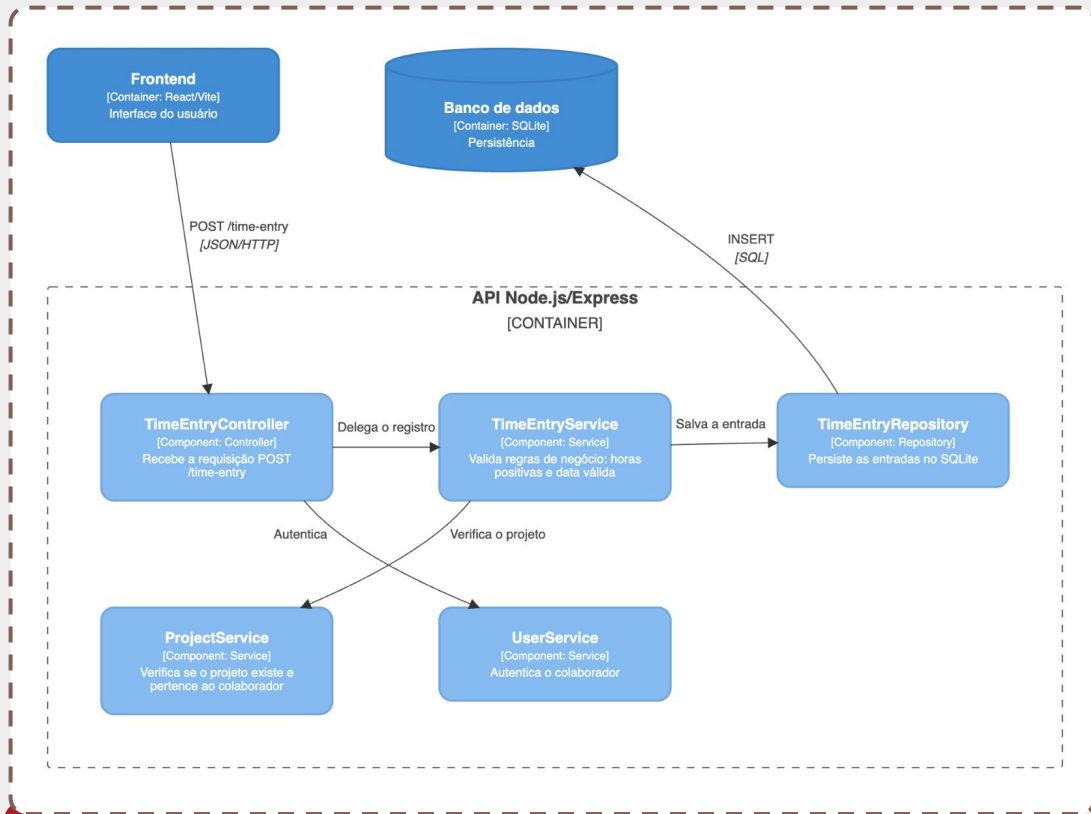
Acessa o banco de dados (ex.: TimeEntryRepository)



No Compasso: **routes/** → **controllers/** → **services/** → **repositories/**



Componente: dentro da API do Compasso



A conexão com as aulas anteriores

Esses componentes **correspondem às pastas das aulas anteriores**. O C4 de Nível 3 formaliza visualmente essa organização.

- ❏ **Dica:** Se a sua estrutura de pastas reflete o diagrama de componentes, você já tem metade da documentação feita.

Compasso: o Diagrama de Componentes (O Mapa do Bairro)

Para a API do Compasso, o Diagrama de Componentes revela:

- **TimeEntryController:** expõe POST /time-entry e trata a requisição HTTP de entrada.
- **TimeEntryService:** valida as regras de negócio (horas positivas, data válida).
- **TimeEntryRepository:** persiste a entrada de horas no SQLite.
- **ProjectService:** verifica se o projeto existe e pertence ao colaborador.
- **UserService:** autentica o colaborador.

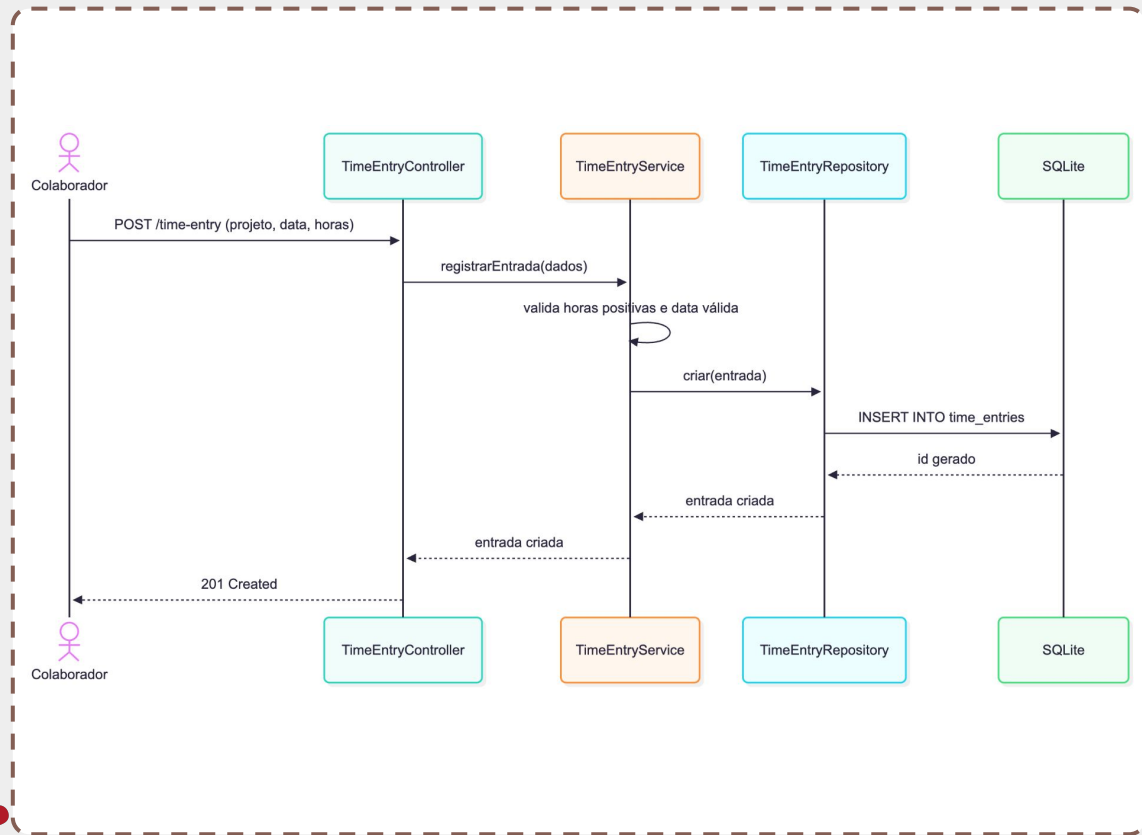
Teste de coesão

Você consegue resumir cada componente em uma frase? Se não, o módulo provavelmente tem coesão baixa.

Os cinco vivem dentro de um único contêiner: a API Node.js/Express.



Os componentes em ação: POST /time-entry



Estrutura × comportamento

O C4 mostra quem existe. O diagrama de sequência mostra quem chama quem em uma requisição real.

Versão simplificada: ProjectService e UserService também participam (slide anterior).

E o Nível 4 — Código?

Raramente documentado

O Nível 4 é o código em si — classes, interfaces, funções. Na prática, **o código é a documentação**.

Se o código precisar de um diagrama para ser entendido, o problema provavelmente é o código, não a falta de diagrama.

Quando faz sentido?

- Algoritmos altamente complexos
- Padrões de design não óbvios
- Integrações muito específicas com bibliotecas externas

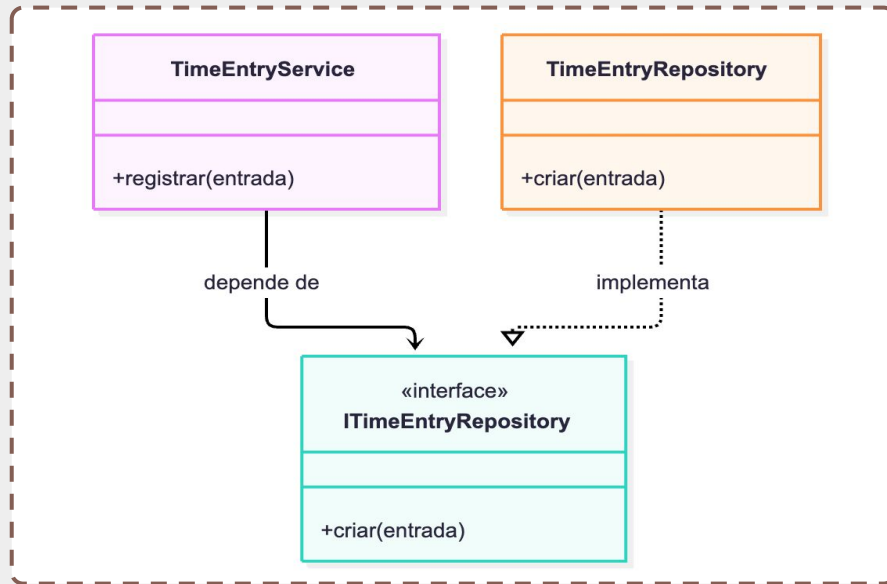
Para o nosso projeto de extensão, **Níveis 1, 2 e 3** são mais do que suficientes.



Compasso: o Diagrama de Código (A Planta Baixa)

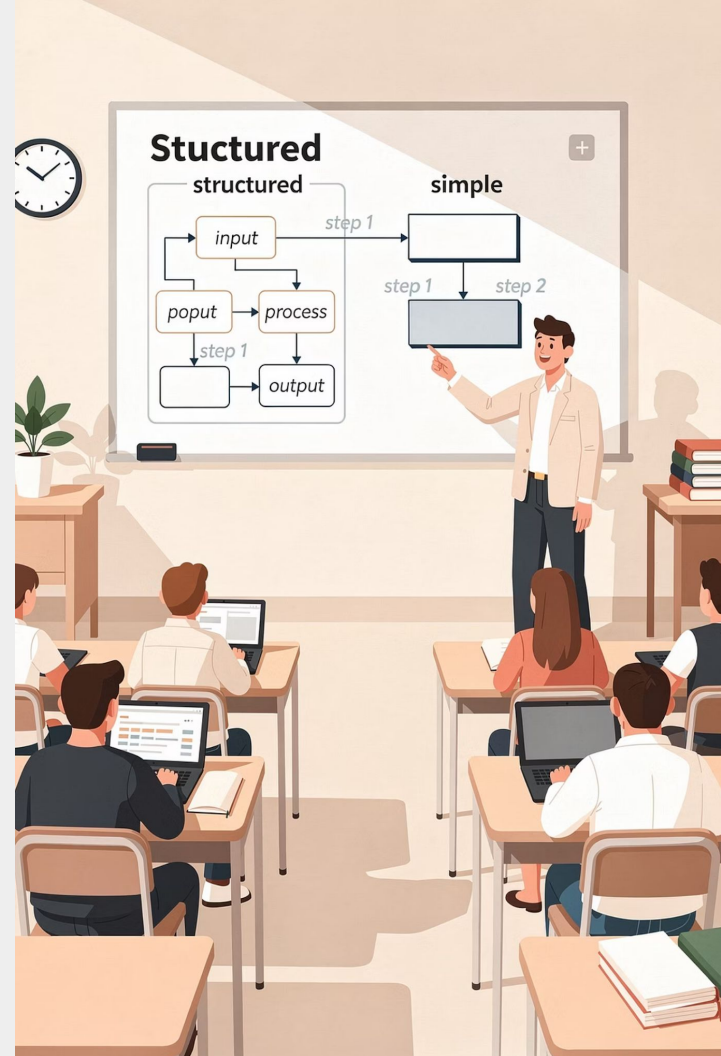
Para o componente TimeEntryService, o Diagrama de Código poderia ser um simples Diagrama de Classes mostrando:

- A classe TimeEntryService.
- Seu principal método público: registrar(entrada).
- Sua dependência de uma abstração (ITimeEntryRepository), em vez de uma implementação concreta ligada ao SQLite, aplicando o Princípio da Inversão de Dependência (DIP) que já discutimos.



C4 vs UML: qual usar?

UML é poderoso para especificação técnica detalhada, mas com 14 tipos de diagramas e notação formal, tem uma curva de aprendizado alta. O C4 tem **4 níveis simples** que qualquer pessoa consegue ler. Para o nosso projeto, C4 é suficiente.



C4 Model vs UML

C4 Model

- 4 níveis intuitivos
- Qualquer pessoa consegue ler
- Foco em comunicação
- Ideal para equipes ágeis
- Ferramenta simples (Mermaid)

UML

- 14 tipos de diagramas
- Notação formal e precisa
- Foco em especificação técnica
- Ideal para projetos com requisitos estáveis
- Ferramentas especializadas (Enterprise Architect, Astah)

Para o nosso projeto de extensão (iterativo, com equipe pequena e arquitetura em evolução) o **C4** é a escolha certa.



Sem mapa, o sistema vira um Big Ball of Mud

Big Ball of Mud (anti-padrão)

- Código desorganizado, sem estrutura clara
- Acoplamento excessivo e baixa coesão
- Difícil de manter e de evoluir
- Cresce sem controle, sob pressão e sem planejamento

O Compasso, com mapa

- Camadas: routes → controllers → services → repositories
- ADRs: SQLite (001) e monolito modular (002)
- Diagramas C4 (N1 a N3) em docs/
- Quem chega sabe onde mexer

Para evitar o Big Ball of Mud: modularização, separação de responsabilidades e uma documentação viva, o mapa que a equipe consulta e atualiza.



Agora é com vocês: rascunho C4 do projeto

Para o Nível 1 – Contexto

- Quem são os atores? Só tem um tipo de usuário ou mais?
- O sistema de vocês se conecta a alguma API externa?
- Existe algum sistema legado ou de terceiros integrado?

Para o Nível 2 – Contêiner

- O frontend e o backend são contêineres separados?
- Qual tecnologia em cada contêiner?
- Qual banco de dados? É o mesmo do exemplo?





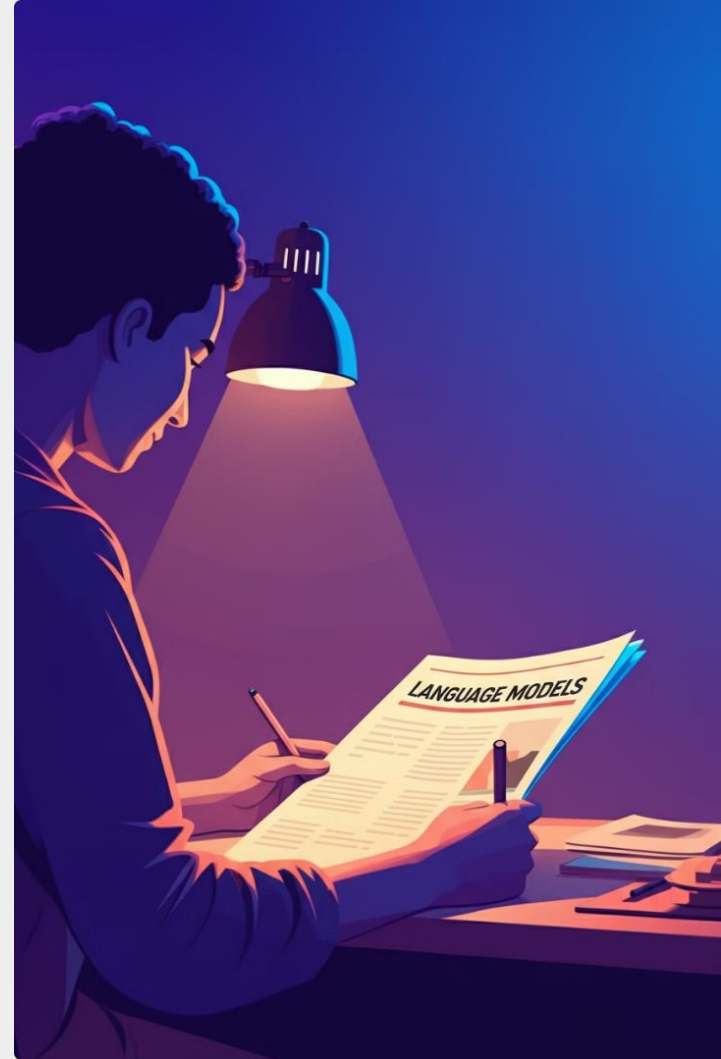
Questão Estruturante

"Considerando que a arquitetura de um software evolui ao longo do desenvolvimento, qual é o valor de documentá-la desde o início do projeto?"



Próximos passos

- Eng. Software em Dimensões
 - Cap. 14 (seções 14.1–14.8):
Arquitetura, C4 Model,
qualidade arquitetural
- Eng. Software Moderna
 - Cap. 5 — Princípios de
Projeto Coesão, acoplamento,
SOLID
 - Cap. 7 — Arquitetura Camadas,
MVC, trade-offs



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
[viniciusgarcia.me](https://www.viniciusgarcia.me)

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Documentação de Arquitetura de
Software